Fetching Data

PYTHON STRATEGIES FOR WEB SERVICES AND WEB SCRAPING

https://github.com/monicaihli/python-data-workshop

Monica Ihli: monica@utk.edu

OBJECTIVES

- Learn to assess use case; Identify & implement appropriate workflow for the situation.
- Learn which libraries + tools will implement the appropriate workflow for a given situation.
- See some simple recipes in action which represent different categories of cases.

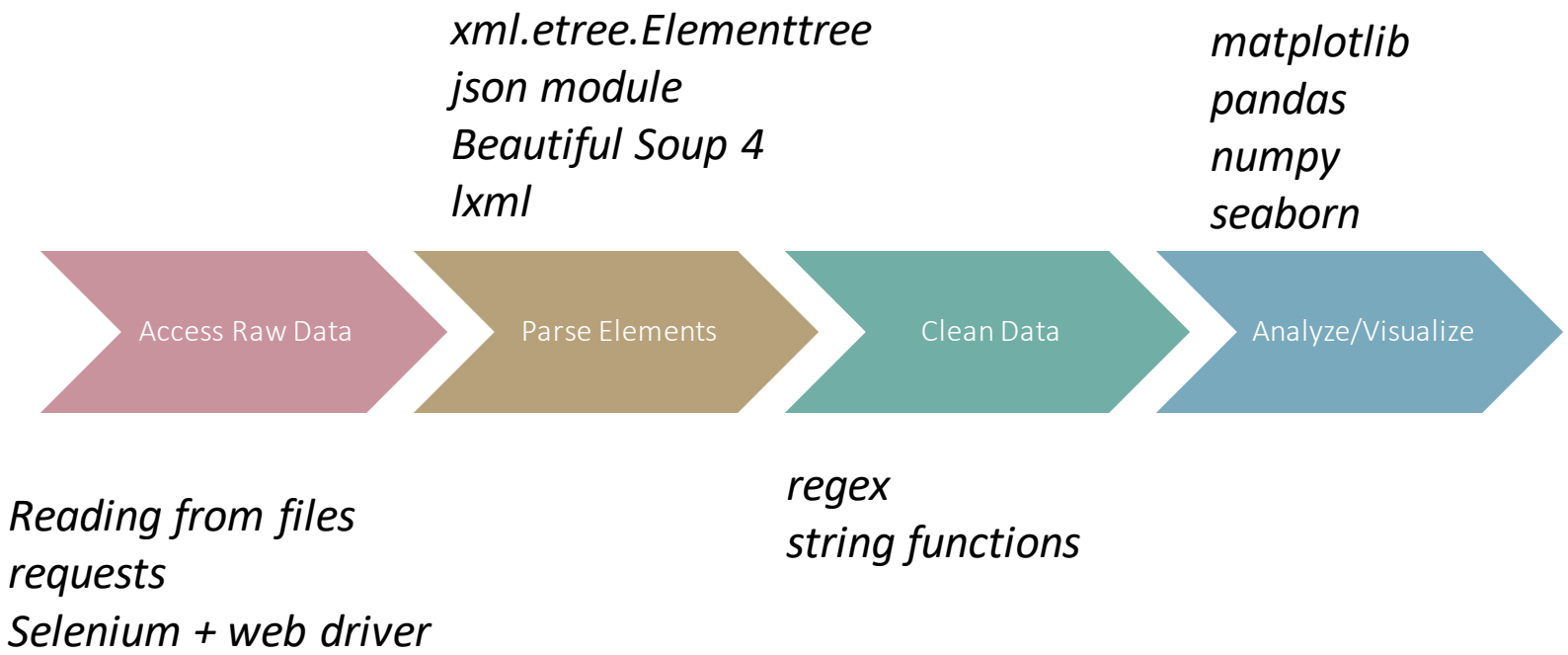
FORMAT

- **Case 1** – Fetch machine readable documents w/ requests.
- **Case 2** – Introducing APIs w/ example.
- *Break*
- **Case 3** – Web scraping data embedded in HTML with requests.
- **Case 4** – Web scraping w/ Selenium + Chromedriver combo.

Data Over HTTP Workflow

Whether you are getting to data through a more structured approach such machine-readable data over an API, or capturing and taming wild HTML to get to information, the basic workflow is the same!

Data Over HTTP Workflow



Getting Data

Human Readable

- Information designed for humans, e.g. web pages.
- Visually appealing.
- Less well-structured.
- Typically more work to extract usable data.

Machine Readable

- Exposed through web services / application programming interfaces.
- Machine readable formats (XML, JSON, even CSV/TSV/etc).
- Well-structured, but ugly to look at.

Human Readable

search.dataone.org/view/doi%3A10.18739%2FA2S17ST4H

Get DataONE Plus NEW Donate Sign-in

DataONE Find Data Services Community Learning About

‹ Back to search | Search / Metadata

Harry McCaughey. 2018. **Carbon dioxide fluxes by the AmeriFlux network at the Ontario - Groundhog River, Boreal Mixedwood Forest site (CA-Gro), 2003-2015.** Arctic Data Center. **doi:10.18739/A2S17ST4H**.

Citations 0 | Downloads 70 | Views 318 | Copy Citation | Assessment report

Files in this dataset Package: resource_map_doi:10.18739/A2S17ST4H

Name		File type	Size		Download All
Metadata: Carbon_dioxide_fluxes_by_the_AmeriFlux_network_at.xml		EML v2.2.0	52 KB		Download
AMF_CA-Gro_BASE_HH_1-1.csv	More info	text/csv	51 MB	8 downloads	Download
AMF_CA-Gro_BIF_LATEST.xlsx	More info	Microsoft Excel OpenXML	25 KB	7 downloads	Download
README_AmeriFlux_BASE.txt	More info	plain text (.txt)	8 KB	55 downloads	Download

General

Machine Readable


```
−<eml:eml packageId="doi:10.18739/A2S17ST4H" xsi:schemaLocation="https://eml.ecoinformatics.org/eml-2.2.0 eml.xsd" scope="system" sy
 −<dataset>
    <alternateIdentifier system="DOI">10.17190/AMF/1245996</alternateIdentifier>
   −<title>
      Carbon dioxide fluxes by the AmeriFlux network at the Ontario - Groundhog River, Boreal Mixedwood Forest site (CA-Gro), 2003-2015
    </title>
   −<creator>
    −<individualName>
        <givenName>Harry</givenName>
        <surName>McCaughey</surName>
      </individualName>
      <organizationName>Queen's University</organizationName>
      <electronicMailAddress>mccaughe@queensu.ca</electronicMailAddress>
      <userId directory="https://orcid.org">https://orcid.org/0000-0003-0896-1255</userId>
    </creator>
    <pubDate>2018</pubDate>
   −<abstract>
      Dynamics of carbon dioxide, water, and energy fluxes and the associated meteorological drivers; assessment of the ecological character of t
      sensing to model carbon dioxide exchange and the status of macronutrients in the canopy.
    </abstract>
   −<intellectualRights>
    −<para>
        This work is licensed under the Creative Commons Attribution 4.0 International License.To view a copy of this license, visit http://creative
      </para>
    </intellectualRights>
   −<distribution>
    −<online>
        <url>http://doi.org/doi:10.18739/A2S17ST4H</url>
      </online>
    </distribution>
   −<coverage>
    −<geographicCoverage>
     −<geographicDescription>
        Groundhog River (FCRN or CCP site 'ON-OMW') is situated in a typical boreal mixedwood forest in northeastern Ontario (48.217 degree
```

cneos.jpl.nasa.gov/ca/

Home | About | Orbits | Close Approaches | Impact Risk | Planetary Defense | Discovery Statistics | Tools | Extras

HOME -> CLOSE APPROACHES -> NEOS

Close Approach Data

The following table shows close approaches to the Earth by near-Earth objects (NEOs) limited as selected in the "Table Settings" below. Data are not available prior to 1900 A.D nor after 2200 A.D. Data are further limited to encounters with reasonably low uncertainty.

Check out our brief video tutorial.

Table Settings: Near future (within 60 days) | Nominal dist. <= 0.05au | no H limit

Show 10 entries

Showing 1 to 10 of 41 entries

Search: Search object

Object	Close-Approach (CA) Date	CA Distance Nominal (LD | au)	CA Distance Minimum (LD | au)	V relative (km/s)	V infinity (km/s)	H (mag)	Estimated Diameter
(2020 TN3)	2020-Oct-13 09:12 ± < 00:01	15.16 | 0.03894	15.06 | 0.03869	18.39	18.38	24.3	38 m - 84 m
(2020 TA3)	2020-Oct-13 10:32 ± 00:07	13.16 | 0.03381	13.02 | 0.03345	6.45	6.44	26.4	14 m - 31 m
(2018 GD2)	2020-Oct-13 13:45 ± 03:59	16.29 | 0.04187	14.44 | 0.03710	6.70	6.69	29.4	3.5 m - 7.8 m
(2020 TD)	2020-Oct-14 14:14 ± < 00:01	18.84 | 0.04841	18.80 | 0.04830	14.41	14.41	24.4	36 m - 80 m
(2020 TO2)	2020-Oct-14 15:08 ± < 00:01	3.17 | 0.00815	3.16 | 0.00812	12.81	12.78	26.6	13 m - 29 m
(2020 TU2)	2020-Oct-14 18:07 ± < 00:01	16.43 | 0.04221	16.32 | 0.04194	13.53	13.53	24.1	41 m - 92 m
(2020 RM6)	2020-Oct-15 05:10 ± < 00:01	12.94 | 0.03325	12.93 | 0.03322	7.70	7.69	25.0	27 m - 61 m
(2020 TE2)	2020-Oct-16 11:15 ± < 00:01	6.37 | 0.01637	6.35 | 0.01633	5.84	5.82	26.4	14 m - 32 m
(2020 TQ3)	2020-Oct-16 12:59 ± 00:01	9.57 | 0.02459	9.52 | 0.02446	5.65	5.63	26.5	13 m - 29 m
(2020 TH3)	2020-Oct-16 15:11 ± < 00:01	4.94 | 0.01270	4.92 | 0.01263	5.14	5.10	26.4	14 m - 31 m

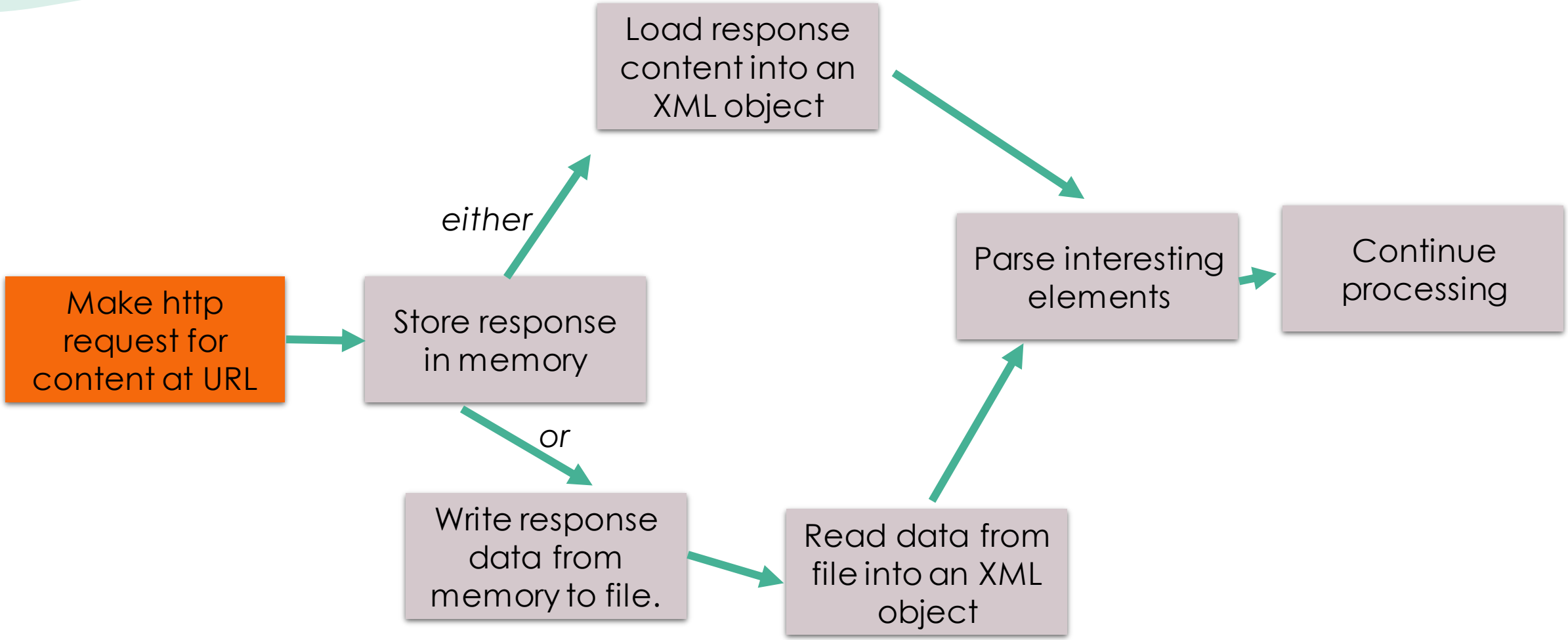
Print | CSV | Excel

Previous 1 2 3 4 5 Next

Use the "Print" button above to print data *contained in this table*. Use the "CSV" or "Excel" buttons to download the data for use in your spreadsheet program. Allow a few seconds for downloads of large datasets.

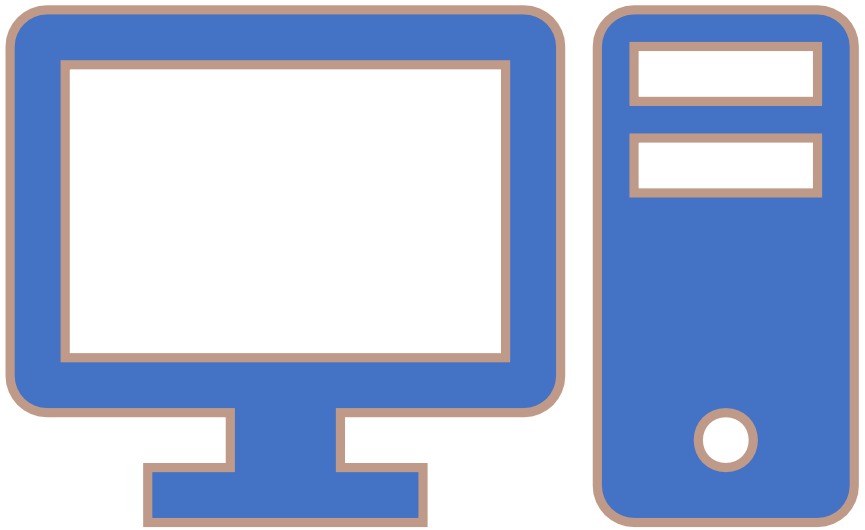
Machine-readable data are available. See the API document for details.

Case I:
Fetch & Parse XML File on Remote Server



Python requests Library

- Third-party library, must install.
- Convenient third-party library which simplifies handling most aspects of working with **HTTP**.



Some HTTP Basics

HTTP Methods indicate the desired action. Examples:

- **GET** – retrieve the requested resource.
- **POST** - Submit data to the resource.
- **PUT** – Update the target resource with the current payload.

"Payload" refers to the data sent over an HTTP request.

HTTP Headers

GENERAL headers like *Date* or *Connection* apply to request or response, but not to the content of what is being sent.

REQUEST headers like Accept, User-Agent

RESPONSE headers like *Age* or *Server* give additional context about the response.

ENTITY headers like *Content-Length, Content-Encoding* describe content of the message body.

Common Status Codes

- 200 – Ok *
- 404 – Not Found
- 503 – Service unavailable
- 500 – Internal Server error

* means no error in http request but problems can happen on other levels!

A simple request

```
import requests
r =
requests.get("http://kitty.ninja")
```

```
r = {Response} <Response [200]>
  _content = {bytes} b'Howdy INSC 360 Cadets!\n'
  _content_consumed = {bool} True
  _next = {NoneType} None
  apparent_encoding = {str} 'ascii'
  connection = {HTTPAdapter} <requests.adapters.HTTPAdapter object at 0x7fa06e1b4748>
  content = {bytes} b'Howdy INSC 360 Cadets!\n'
  cookies = {RequestsCookieJar} <RequestsCookieJar[]>
  elapsed = {timedelta} 0:00:00.188703
  encoding = {str} 'ISO-8859-1'
  headers = {CaseInsensitiveDict} {'Date': 'Wed, 25 Mar 2020 20:11:12 GMT', 'Server': 'Apache/2
  history = {list} <class 'list'>: []
  is_permanent_redirect = {bool} False
  is_redirect = {bool} False
  links = {dict} <class 'dict'>: {}
  next = {NoneType} None
  ok = {bool} True
  raw = {HTTPResponse} <urllib3.response.HTTPResponse object at 0x7fa06e1db240>
  reason = {str} 'OK'
  request = {PreparedRequest} <PreparedRequest [GET]>
  status_code = {int} 200
  text = {str} 'Howdy INSC 360 Cadets!\n'
  url = {str} 'http://kitty.ninja/'
```

Other HTTP Request Types

```
>>> r = requests.put('https://httpbin.org/put', data =
{'key':'value'})

>>> r = requests.delete('https://httpbin.org/delete')

>>> r = requests.head('https://httpbin.org/get')

>>> r = requests.options('https://httpbin.org/get')
```

Request Arguments

- **<u>url</u>**- You can provide a domain-based or even an IP address:

```
url='http://167.99.155.69')
```

- **<u>params</u>** – a dict of key-value pairs representing HTTP queries and inputs:

```
payload = {'verb': 'ListRecords', 'from': '2020-03-25' }
```

Request Parameters

- **headers** – constructed using a dict of key-value pairs:

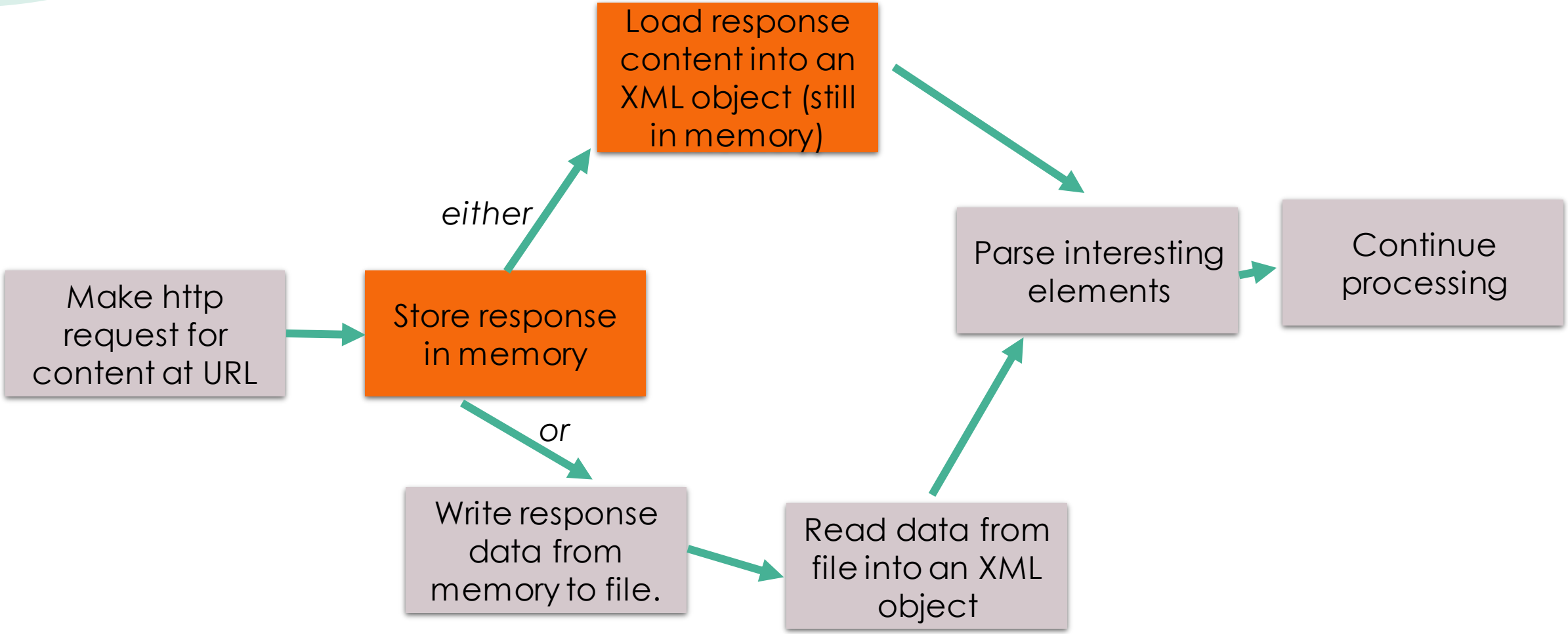
```
headers = {'user-agent': 'Mozilla/5.0 (X11; Ubuntu; Linux x86_64;
rv:74.0) Gecko/20100101 Firefox/74.0'} # haha look like a human!
```

```
r = requests.get(url=baseurl, params=payload, headers=headers)
```

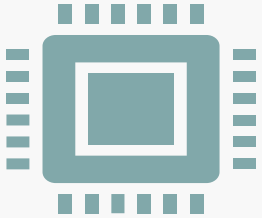
case-1_A.py

- Import requests library.
- Use requests to make an HTTP response to a web server. (http://volweb.utk.edu/~mihli1/demos/page1.html)
- Check response code for expected outcome.
- Access the contents of the server's response (the page you requested).
- View response headers.

Case I:
Fetch & Parse XML File on Remote Server



Computer Storage: RAM vs ROM



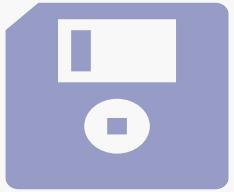
RAM

Random Access Memory

Temporary storage

Data disappears when the power goes

When we create a variable in Python, we are allocating it temporary space in RAM.



ROM

Read Only Memory

Permanent storage

Data is stored on disk even when power is off

When we save data to a file, we are allocating memory on disk in ROM.

XML Tools

- XML: eXtensible Markup Language
- Standard Library XML Parser (we use today): *xml.etree.ElementTree*
- Other library options:
  - *lxml* – speedy parsing.
  - *BeautifulSoup* - mainly famous for HTML parsing but also works with an implementation of *lxml.*
  - *xml.dom.minidom* - minimal DOM. Also gives us pretty XML.

`xml.etree.ElementTree`

- Loads data into a tree-like structure in memory.
- Facilitates both searching for specific elements of data in a document, or iteratively accessing all elements.
- Can modify an existing XML tree , create a new one, and even write XML trees to file.

xml.etree.ElementTree

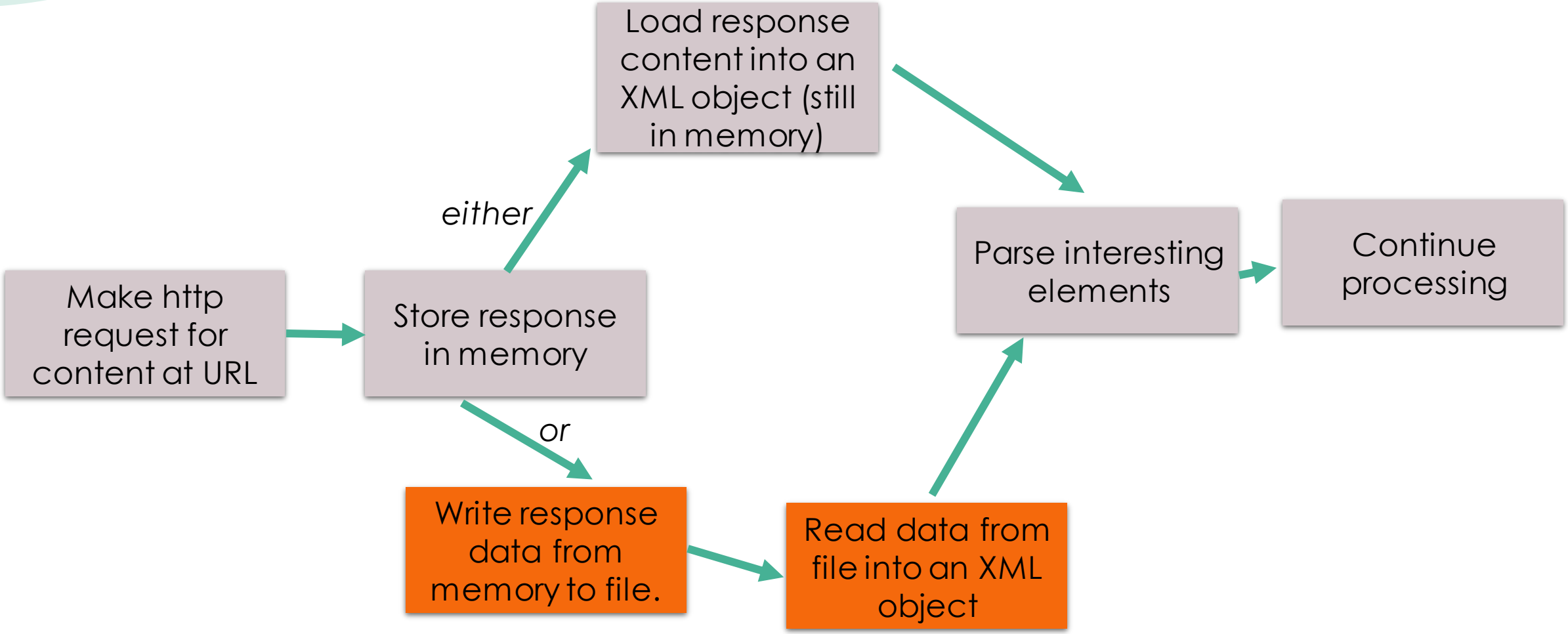
```
# creates a new XML object in memory from data already in memory as string
xml_data = ET.fromstring(r.content)

# creates new XML object in memory by reading from data stored on disk as a file
Xml_data = ET.parse("demo1.xml")
```

case-1_B.py

- Fetch an XML file off a remote server using requests library. (http://volweb.utk.edu/~mihli1/demos/case1.xml)
- Load the contents of the XML document into an element tree object.
- Display the XML elements.

Case 1:
Fetch & Parse XML File on Remote Server



case-1_C.py

- Fetch a set of XML files off a remote server using requests library. (http://volweb.utk.edu/~mihli1/demos/case1-C/file1.xml)
- Write the files to disk
- Cycle through all downloaded XML files in a directory and display them.
- Display the XML elements.

Understanding XML & Parsing w/ Python

Anatomy of a Simple XML Document

```
<?xml version = "1.0" encoding = "UTF-8" ?>
<snacks>
    <cookies>
        <cookie>chocolate chip</cookie>
        <cookie>oatmeal raisin</cookie>
        <cookie>peanut butter</cookie>
    </cookies>
    <pies>
        <pie>apple</pie>
        <pie>pecan</pie>
        <pie>pumpkin</pie>
    </pies>
</snacks>
```

Document prolog

Anatomy of a Simple XML Document

```
<?xml version = "1.0" encoding = "UTF-8" ?>
<snacks>
    <cookies>
        <cookie>chocolate chip</cookie>
        <cookie>oatmeal raisin</cookie>
        <cookie>peanut butter</cookie>
    </cookies>
    <pies>
        <pie>apple</pie>
        <pie>pecan</pie>
        <pie>pumpkin</pie>
    </pies>
</snacks>
```

root element

Anatomy of a Simple XML Document

```
<?xml version = "1.0" encoding = "UTF-8" ?>
<snacks>
  <cookies>
      <cookie>chocolate chip</cookie>
      <cookie>oatmeal raisin</cookie>
      <cookie>peanut butter</cookie>
  </cookies>
  <pies>
      <pie>apple</pie>
      <pie>pecan</pie>
      <pie>pumpkin</pie>
  </pies>
</snacks>
```

example of opening tag

example of closing tag

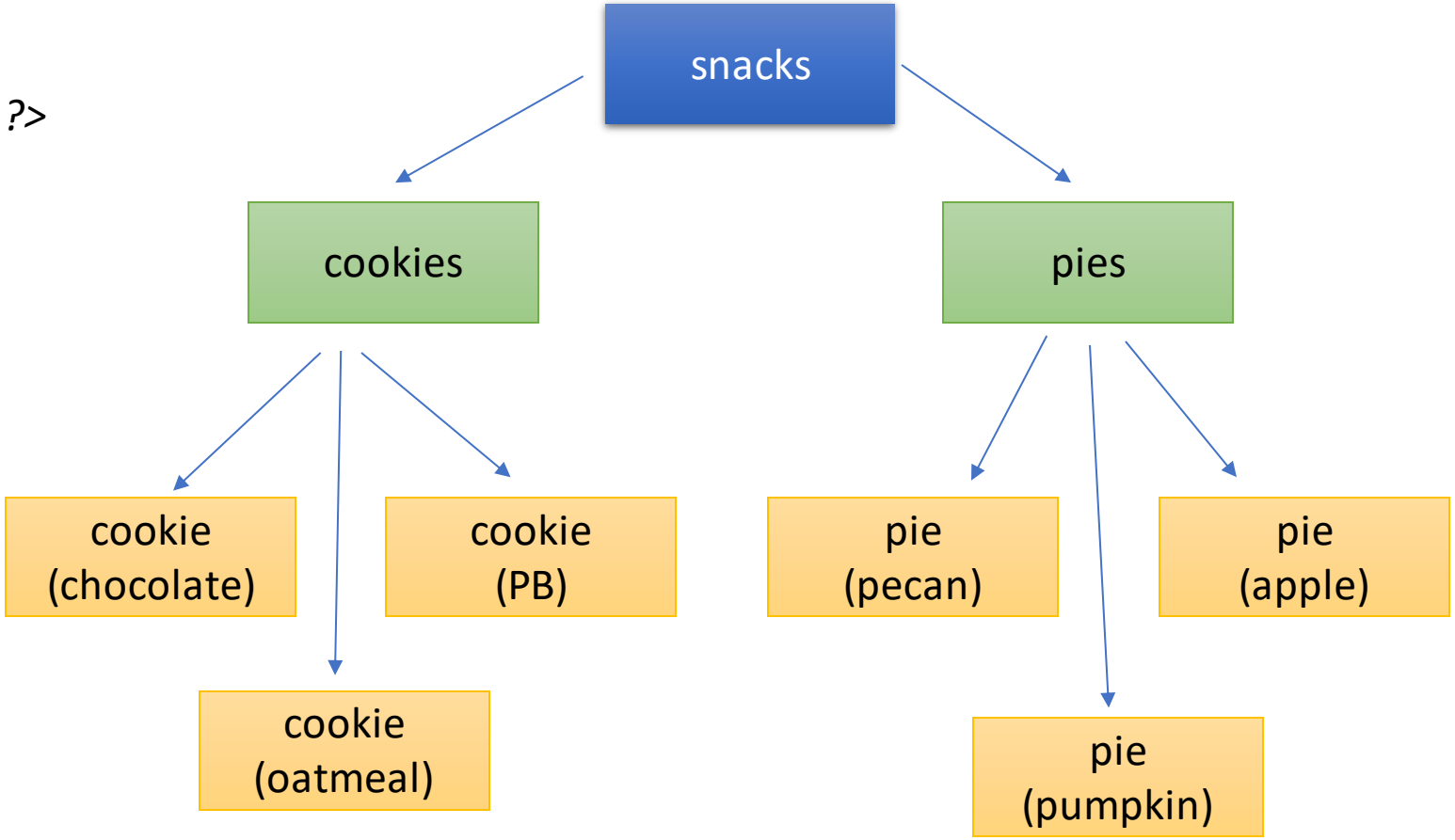
Elements with Attributes

```
<?xml version = "1.0" encoding = "UTF-8" ?>
<snacks>
    <cookies>
        <cookie sellby="03/14/2020">chocolate chip</cookie>
        <cookie sellby="03/13/2020">oatmeal raisin</cookie>
        <cookie sellby="03/15/2020">peanut butter</cookie>
    </cookies>
    <pies>
        <pie sellby="03/5/2020">apple</pie>
        <pie sellby="03/9/2020">pecan</pie>
        <pie sellby="03/6/2020">pumpkin</pie>
    </pies>
</snacks>
```

- Attributes can be used to store additional information or properties about the element.
- Each attribute can contain one value.

Structure of a Simple XML Document

```
<?xml version = "1.0" encoding = "UTF-8" ?>
<snacks>
    <cookies>
        <cookie>chocolate chip</cookie>
        <cookie>oatmeal raisin</cookie>
        <cookie>peanut butter</cookie>
    </cookies>
    <pies>
        <pie>apple</pie>
        <pie>pecan</pie>
        <pie>pumpkin</pie>
    </pies>
</snacks>
```



`xml.etree.ElementTree`

- Module has two classes for representing the XML data;
  - **ElementTree** – represents the whole document as a tree structure
  - **Element** – represents a single node in the tree (But this node can have children)

- Data can be imported from a string (in memory) or from a file.

- An element can consist of:
  - **tag** – a string identifying the name of the element.
  - **text** – the text contained within the start and end tag, if any.
  - **tail** – in some situations, to capture additional text outside the tag.
  - **attributes** – attributes are stored as a dictionary where the attribute label is the key and the value is the... value.

Strategies for Parsing XML Elements

(1) Iteratively Drilling Down – Use for loops to iterate over every child of a node, regardless of what the tag is. Repeat with additional nested for loop to drill further down the hierarchy.

(2) Drill down, one level at a time, through successive rounds of .find()/.findall() to select specific nodes which match a pattern. By default, the patterns only search immediate children.

(3) Use .find()/.findall() but provide an xpath search pattern to navigate straight to the desired node.

Searching a Tree: .find() and .findall()

- **.find()** - Finds the first child matching by tag name or path.
- **.findall()** - Finds all matching children, Returns as list.
- **.find()/.findall()** will only search immediate children unless an xpath is used: https://docs.python.org/3/library/xml.etree.elementtree.html#supported-xpath-syntax
- If no matches can be found, then .find() will return *None* type, and .findall() will return an *empty list*.

Accessing Attributes

```
<?xml version = "1.0" encoding = "UTF-8"?>
<snacks>
    <cookies>
        <cookie sellby="03/14/2020">chocolate chip</cookie>
        <cookie sellby="03/13/2020">oatmeal raisin</cookie>
        <cookie sellby="03/15/2020">peanut butter</cookie>
    </cookies>
</snacks>
```

```
import xml.etree.ElementTree as ET
tree = ET.parse("demo1.xml")
root = tree.getroot()

list_of_cookies = root.findall(".//cookie")
for cookie in list_of_cookies:
    print(cookie.text)
    print(cookie.get("sellby"))
```

Output:

```
chocolate chip
03/14/2020
oatmeal raisin
03/13/2020
peanut butter
03/15/2020

Process finished with exit code 0
```

Searching Attributes

```
<?xml version = "1.0" encoding = "UTF-8" ?>
<snacks>
    <cookies>
        <cookie sellby="03/14/2020">chocolate chip</cookie>
        <cookie sellby="03/13/2020">oatmeal raisin</cookie>
        <cookie sellby="03/15/2020">peanut butter</cookie>
    </cookies>
</snacks>
```

```
import xml.etree.ElementTree as ET
tree = ET.parse("demo1.xml")
root = tree.getroot()
# notice how the inner value of the attribute in the search is inside single quotes
this_cookie = root.find(".//cookie[@sellby='03/15/2020']")
print("Cookie batch with sell by date of 03/15/2020: " + this_cookie.text)
```

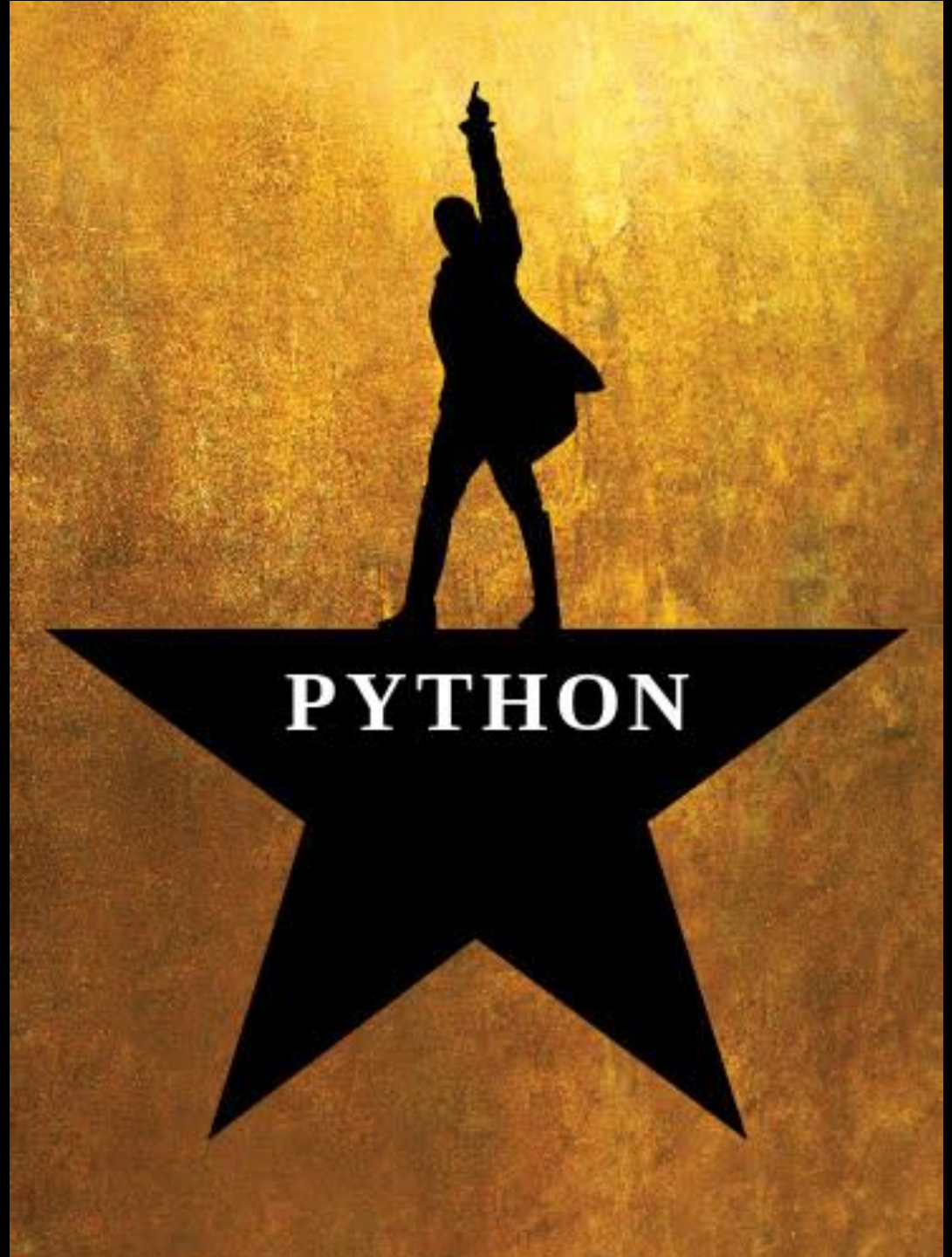
Output:

```
Cookie batch with sell by date of 03/15/2020: peanut butter

Process finished with exit code 0
```

Case 2: Fetching Data Over an API

(application programming interface)



1

API – Functionality for letting two kinds of software talk to each other.

2

Web Services – An implementation of an API for transferring data over the web

3

You will often hear these terms used **interchangeably**.

Dealing with Jargon

API Examples

- NASA Open APIs:
  https://api.nasa.gov/

- PubMed:
  https://www.ncbi.nlm.nih.gov/home/develop/api/

- Elsevier Scopus / Science Direct/ SciVal (Scholarly literature and journal articles):
  https://dev.elsevier.com/

- Library of Congress:
  https://labs.loc.gov/lc-for-robots/

- Twitter:
  https://developer.twitter.com/en/doc

More Jargon

- An API **request** will start with a **base url** (hostname + api path)
- An API can have multiple **endpoints** -- the locations from which you request different resources.
- Request is fleshed out with **parameters** that tell the server the specific information you are asking for.
- A **rate limit** or **quota** refers to how many requests you are allowed to make in a certain amount of time.
- Some services require you to aquire an **API Key** (secret passcode) to use the service.

Anatomy of a Web Services Request: Elsevier

https://api.elsevier.com/content/search/scopus?apiKey=my_api_key&query=AF-ID(60024266) AND PUBYEAR = 2018

Base url: https://api.elsevier.com/content/search

Endpoint: /scopus

Parameters:

apiKey=my_api_key

query=AF-ID(60024266) AND PUBYEAR = 2018

Anatomy of a Web Services Request: Elsevier

https://api.elsevier.com/content/search/scopus?apiKey=my_api_key&query=AF-ID(60024266) AND PUBYEAR = 2018

? separates the base url and endpoint from query parameters.

& separates multiple parameters from each other. It designates the end of one parameter and the start of another.

Anatomy of a Web Services Request: Pubmed OAH-PMH

https://www.ncbi.nlm.nih.gov/pmc/oai/oai.cgi?verb=ListRecords&from=2019-02-01
&until=2019-10-01&set=bmcbioc&metadataPrefix=oai_dc

Base url: https://www.ncbi.nlm.nih.gov/pmc *(API I want to use)*

Endpoint: /oai/oai.cgi *(The resource communication point I'm interested in)*

Parameters:

verb=ListRecords *(From OAI-PMH standard, what I'm asking for)*

from=2019-02-01 *(Start date for interval of records I want)*

until=2019-10-01 *(End date for the records I'm interested in)*

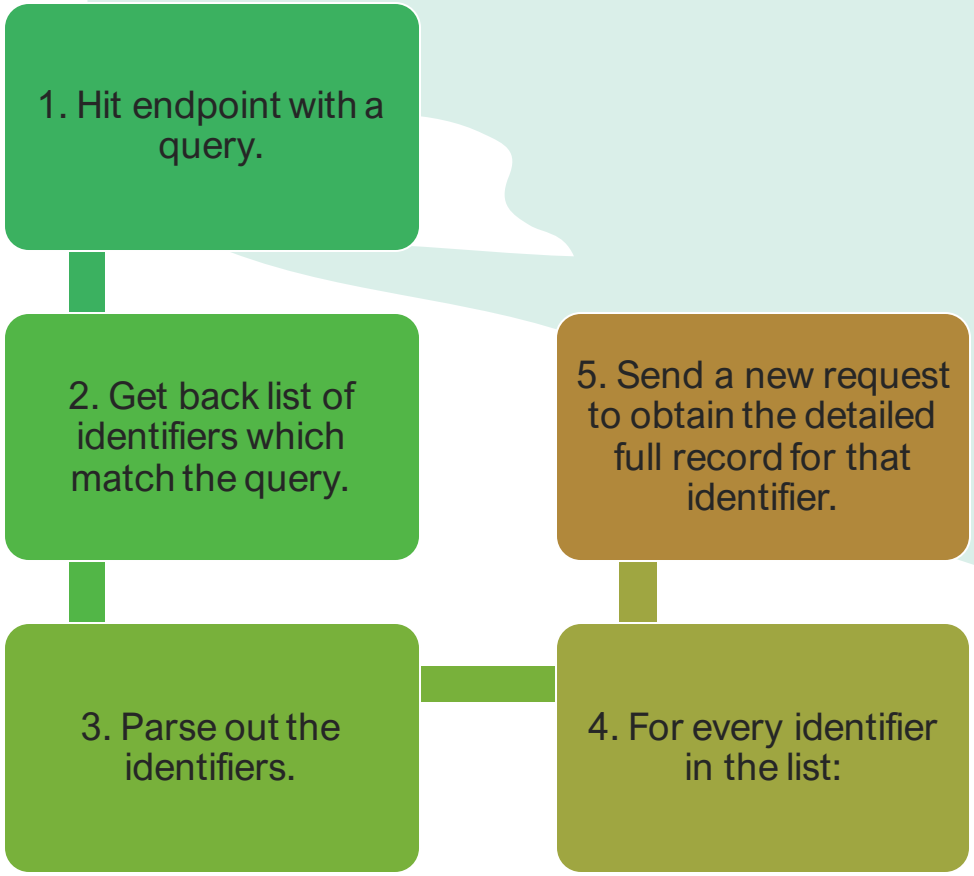
set=bmcbioc *(Only records in this category)*

metadataPrefix=oai_dc *(The metadata format I want results in)*

Some Answers to find in an API's Documentation

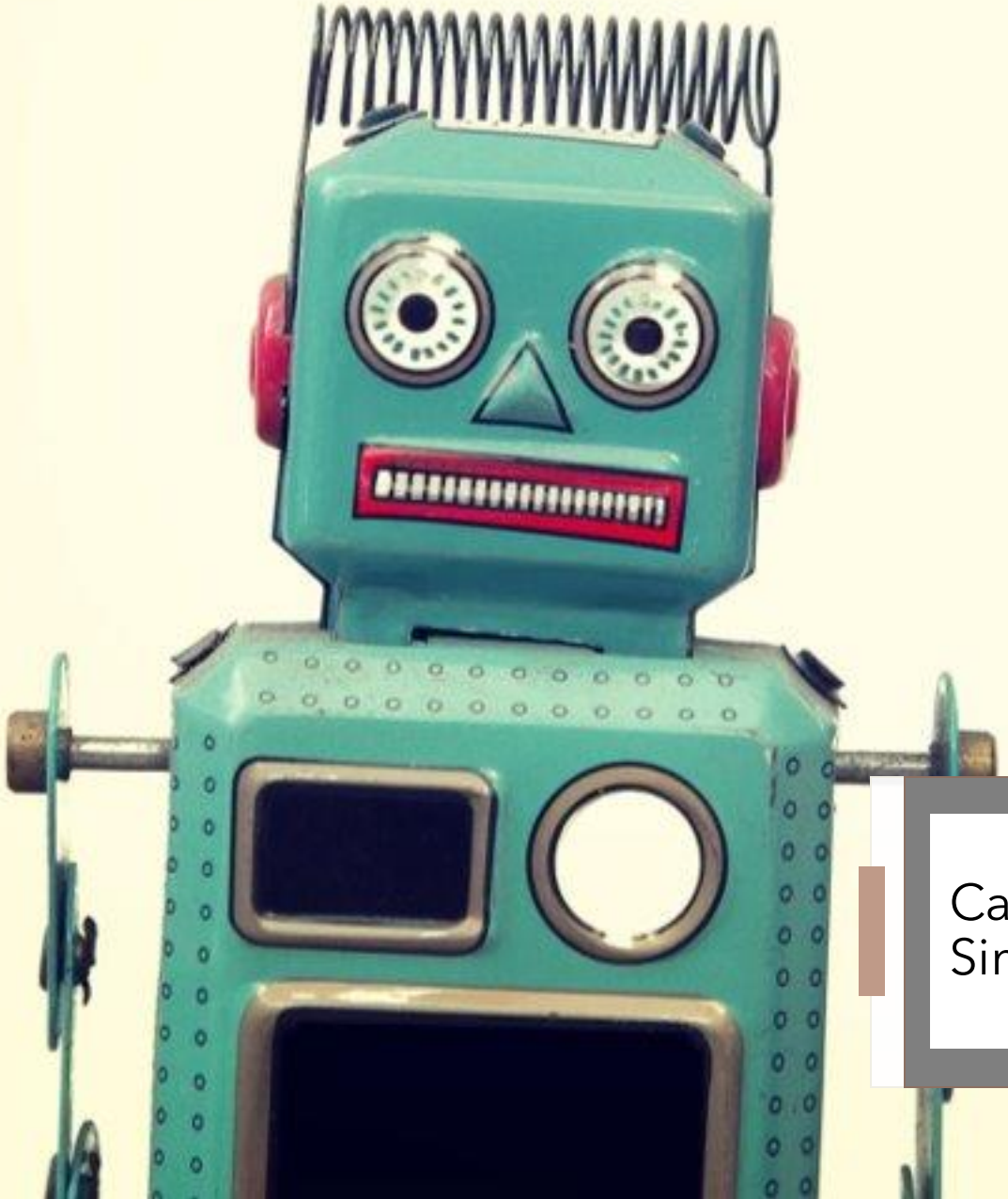
- Is there authentication and how does it work?
  - API Key?
  - IP-restricted?
- What are the major end points and what are their parameters?
- Is there a rate limit / quota?
- How does pagination work?
  - How many records per request.
  - How to cycle through all records that match your query when the number of results exceeds the number that you can fetch at one time.

A Common Workflow for APIs



case-2.py

- Use NASA public API to capture data about near earth objects.



Case 3
Simple Web Scraping

Requests
+ Beautiful Soup

Web Scraping

- Web scraping usually refers to extracting data from web pages, often in a fashion that simulates human behavior.

- Web scraping often entails heavy reliance on string-processing functions to clean data extracted from HTML, which is far more unruly and wild than the neatly structured machine-readable content exposed in APIs.

About Beautiful Soup 4

- To parse a document, pass either a string or a file handler to the BeautifulSoup constructor
- Content is first converted to Unicode.
- Transforms a complex HTML document into a complex tree of Python objects.
- The 4 objects you will interact with are:
  - **Tag** – Corresponds to an html or xml element; Has name and attributes
  - **NavigableString** – Contains text within a tag
  - **BeautifulSoup** – represents the parsed document as a whole
  - **Comment** – special type of NavigableString

https://www.crummy.com/software/BeautifulSoup/bs4/doc/

bs4 parser options

This table summarizes the advantages and disadvantages of each parser library:

Parser	Typical usage	Advantages	Disadvantages
Python's html.parser	`BeautifulSoup(markup, "html.parser")`	• Batteries included • Decent speed • Lenient (As of Python 2.7.3 and 3.2.)	• Not as fast as lxml, less lenient than html5lib.
lxml's HTML parser	`BeautifulSoup(markup, "lxml")`	• Very fast • Lenient	• External C dependency
lxml's XML parser	`BeautifulSoup(markup, "lxml-xml")` `BeautifulSoup(markup, "xml")`	• Very fast • The only currently supported XML parser	• External C dependency
html5lib	`BeautifulSoup(markup, "html5lib")`	• Extremely lenient • Parses pages the same way a web browser does • Creates valid HTML5	• Very slow • External Python dependency

https://www.crummy.com/software/BeautifulSoup/bs4/doc/

BS4: Navigating the Tree

- Tags may contain strings and other tags as children.
- Can navigate using tag names (such as `soup.p` or `soup.title`)
- Can access tag attributes similarly to a dictionary. (`tag['id']`)
- `soup.find_all('p')` <- Would return a set of tags which match p (paragraph)
- All a tags childrens are available in a list as a property of the tag called .contents
- Can also access children of a tag through a generator:
  `for child in title_tag.children:`

https://www.crummy.com/software/BeautifulSoup/bs4/doc/

Parse Elements

```
for child in soup.children:
    print(child)
```

```
<!DOCTYPE html>
<html lang="en">
<head>…  </head>
<body>
   <h2>Top</h2>
   <p class="top">This is the first paragraph</p>
   <p class="top">This is the second
paragraph</p>
   <h2>Middle</h2>
   <p>This is the <b>middle</b> paragraph</p>
   <h2>Bottom</h2>
   <p class="bottom">This is the fourth
paragraph</p>
   <p class="bottom">This is the fifth
paragraph</p>
</body>
</html>
```

Parse Elements

```
paragraphs = soup.find_all('p')
for p in paragraphs:
    print(p.text)
    if p.b:
        print("{} is in BOLD!".format(p.b))
```

```
<!DOCTYPE html>
<html lang="en">
<head>...  </head>
<body>
   <h2>Top</h2>
   <p class="top">This is the first paragraph</p>
   <p class="top">This is the second
paragraph</p>
   <h2>Middle</h2>
   <p>This is the <b>middle</b> paragraph</p>
   <h2>Bottom</h2>
   <p class="bottom">This is the fourth
paragraph</p>
   <p class="bottom">This is the fifth
paragraph</p>
</body>
</html>
```

case-3.py

- Introduction to web-scraping a sample file: http://volweb.utk.edu/~mihli1/demos/case3.html
- Obtain web page using requests.
- Scrape sample content with beautiful soup and HTML parser.

Web Scraping with Selenium

- Selenium is a package for automated testing of web pages. It replicates user behavior like clicking, filling out form data, pausing, etc.
- It has also become quite popular for web scraping due to reasons such as ability to interact with pages dynamically, allow JavaScript to execute.

Capturing Raw Data Over HTTP

REQUESTS

- Faster capture over HTTP.
- Simpler to implement.
- Can't interact with page.
- Won't run javascript.
- No default user-agent.
- Best choice for web services.

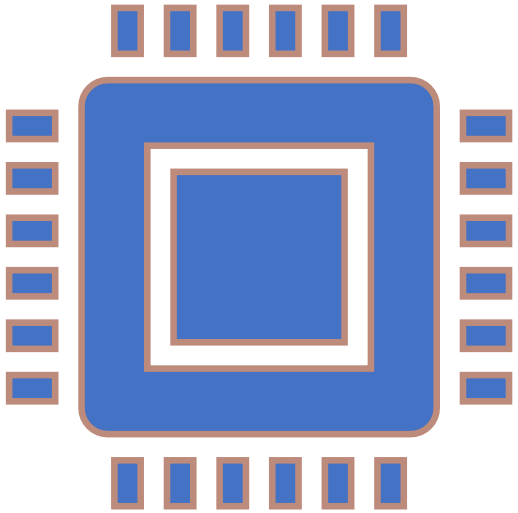
SELENIUM/WEB DRIVER

- Can interact with webpage to get to desired data such by navigating and clicking buttons.
- Will execute JavaScript.
- Slower than requests because you are operating through a browser.
- Browser's default user-agent
- Best choice for web pages with JS or needing interaction.

Web Scraping w/ Selenium

- Sometimes web pages aren't fully rendered in the moment you make the request. A page with JavaScript can take time to finish processing and loading the content.
- This means that whatever you get back from that HTTP request isn't going to have the full page content.
- Solution: Use Selenium & chromedriver-- Take control of a browser (Chrome) programmatically using a special driver.
- Allows page to fully load before retrieving its contents.

Web Scraping: The Complete Package



- **Selenium** – Python package implements tools for controlling a web driver.
- **Chromedriver** (or other web driver) - tool for automated testing of webapps. It supports navigating to web pages, user input, executes JavaScript, etc.
- **Google Chrome** – It is possible with other browsers but this is what we will use.
- **Beautiful Soup** – a Python library for pulling data out of HTML and *XML files.

** Note: If you want to use for XML parsing, will need to install lxml package*

Create Web Driver & Fetch Page Source

```
url = "http://kitty.ninja/demo.html"
driver = webdriver.Chrome(executable_path="~/Downloads/webdrivers/chromedriver")
driver.get(url)
```

Store Document Tree in Memory

```
soup = BeautifulSoup(driver.page_source, "html.parser")
```

case-4.py

- Example of scraping web page that uses javascript: http://volweb.utk.edu/~mihli1/demos/case4.html
- Launch chromedriver using selenium to control browser and interact with page.
- Load the source code of the browser being controlled into beautiful soup and proceed to parse as usual.